
EECS 545 Machine Learning

Project Proposal

GNN on Traveling Salesman Problem

Pingbang Hu

Department of Computer Science
University of Michigan
pbb@umich.edu

Jonathan Moore

Department of Computer Science
University of Michigan
moorejon@umich.edu

Yi Zhou

Department of Math
University of Michigan
yizho@umich.edu

Shubham Kumar Pandey

Department of Statistics
University of Michigan
pandeysk@umich.edu

Anuraag Ramesh

Department of Statistics
University of Michigan
anuraagr@umich.edu

Abstract

This is a project proposal of EECS545 at the University of Michigan. We'll formulate the problem about TSP, show the significance of our potential approaches, briefly discuss some related works, and bring up some ideas which can potentially push the limit of this well-studied problem forward.

Keywords: Travelling salesman problem, TSP, Graph Neural Network, Deep Learning, Reinforcement Learning, Embedding learning, Link prediction, Combinatorial Optimization, Learning theory.

1 Problem Statement

The travelling salesman problem, also called the travelling salesperson problem or TSP, is a problem as follows. Given a list of cities and the distances between each pair of cities, we want to find the shortest possible route that visits each city *exactly once* and returns to the origin city.

Specifically, given an **undirected weighted graph** $\mathcal{G} = (\mathcal{E}, \mathcal{V})$, which is an ordered pair of nodes set $\mathcal{E}$ and edges set $\mathcal{V} \subseteq \mathcal{E} \times \mathcal{E}$ where $\mathcal{G}$ is equipped with **spatial structure**. This means that each edge between nodes will have different weights and each node will have its coordinates, we want to find a simple cycle that visits every node exactly once while having the smallest cost.

2 Significance

Because of the expressive power of graphs, developing and analyzing graphs using machine learning has become a rising topic in computer science. In recent years, graph neural networks (GNN), among its variants, have demonstrated significant potential in many areas.[5] The problems that GNN can solve are classified into the following categories: Node/Link Classification, Graph Classification, Graph Visualization, Node/Link Prediction, and Graph Clustering. Its applications have been promising in fields including computer vision, natural language processing, chemistry, recommended systems, traffic, and social/brain networks.

We expect our project to further demonstrate the power of GNN, especially its ability to handle large datasets, through applying it to the Travelling Salesman Problem. Since TSP has been one of the most intensively studied problems in optimization, numerous solutions have been developed over

the years. Most of the solutions use either the exact algorithm, which considers all permutations and gives an optimal path, or the heuristic and approximation algorithms, which yields a path that is not necessarily optimal, but pretty good in a shorter time. The two algorithms have shown promising results in finding a short path. However, their running time becomes significant as the dataset grows larger. Our approach will potentially demonstrate how GNN can handle large datasets better than some of the traditional methods. We could also show that, with some more work, GNN could provide promising solutions to both classical and recent computer science problems.

3 Related Work

There has been a lot of progress on the symmetric TSP in the last century, as it is one of the classical NP-hard combinatorial problems. With the increase in the number of nodes, there is a superpolynomial (at least exponential) explosion in the number of potential solutions. This makes the TSP problem difficult to solve on two parameters, the first being finding a global shortest route as well as reducing the computation complexity in finding this route.

The state-of-the-art TSP solvers such as Concorde¹ are exact solvers, these solvers basically rely on crafting handful heuristics and approximating the solution. Concorde uses a plane-cutting method, which tries to reduce the solution space. It has been shown that solving TSP requires a combination of two algorithms to approximate the shortest path. One algorithm is needed to approximate the set of heuristics to traverse the solution space efficiently. The second algorithm uses heuristics and performs an ad-hoc search[2].

There are two main types of methods to find the required heuristic for the TSP problem: autoregressive and non-autoregressive deep learning. Autoregressive methods are aggressive in their nature, i.e., the updates take place after every iteration. Inversely, non-autoregressive methods are highly parallelized and combine individual results at the end. Studies suggest that non-autoregressive deep learning methods work better for NP-hard combinatorial problems like TSP[2]. The structured and non-trivial nature of the optimization problem makes it suitable for neural networks[2] utilizes a Graph Convolution Network followed by a post-ad-hoc beam search algorithm. The above approach when used for a small graph (20 nodes) produces a result of 0.10% gap from the optimal Concorde solver and the computation is 33% faster.

[3] suggests an alternative to large-scale training computational costs by learning from small-scale TSP and transferring the learned model in a zero-shot fashion on larger graphs. Our proposed method tries to implement these ideas by applying GNN in combination with Deep neural networks. [1] justifies this approach by comparing its advantages to existing neural network methodology in a more theoretical context. We implement this approach more specifically to the TSP.

4 Proposed Method

From the existing papers and some state-of-the-art algorithms suggest, we proposed the following potential methods to tackle TSP under GNN.

1. Semi-supervised learning technique under GNN.
2. GNN with Deep Neural Network (DNN).
3. GNN with Reinforcement Learning (RL).
4. GNN with labeling trick[4]. This includes two node embedding (essentially will be used on link prediction), subgraph embedding, and so on. The algorithm we'll potentially use is SEAL².
5. Link prediction by node prediction technique on GNN.

Besides training methods, since TSP can be formulated as *Integer programming*³, we will (potentially) investigate some theoretical properties about *Integer programming* by optimization theory, under the

¹<https://www.math.uwaterloo.ca/tsp/concorde.html>

²<https://github.com/muhanzhang/SEAL>

³https://en.wikipedia.org/wiki/Integer_programming

setup of GNN. Notice that since we care about the **generalization ability** essentially, this part will not be addressed as much as others.

Lastly, since the generalization ability of GNN is hugely impacted by the data sampling method we used for generating training data, hence we will put our focus of theoretical analysis on this point, namely to find the best way to generate the training data given a usual input of TSP coordinate and the optimal path.

5 Evaluation

Our primary way of evaluating our success will be by measuring how close to the optimal solution our models get after various amounts of time compared to more traditional methods. For example, we would consider this project a success if our model could, on average, find a path $1.2\times$ the minimum length in an hour when a more traditional model, on average, could find a path $1.25\times$ the minimum length in the same amount of time (on the same hardware, etc.).

For a more objective goal, we will measure our success based on the relative percentage over the minimum compared to other models. In the previous example, we would score $.2/.25 = .8$. The closer the 0 the better, and any value less than 1 represents a success.

References

- [1] Paul Almasan, José Suárez-Varela, Arnau Badia-Sampera, Krzysztof Rusek, Pere Barlet-Ros, and Albert Cabellos-Aparicio. Deep reinforcement learning meets graph neural networks: exploring a routing optimization use case, 2020.
- [2] Chaitanya K. Joshi, Thomas Laurent, and Xavier Bresson. An efficient graph convolutional network technique for the travelling salesman problem. *CoRR*, abs/1906.01227, 2019. URL <http://arxiv.org/abs/1906.01227>.
- [3] Chaitanya K Joshi, Quentin Cappart, Louis-Martin Rousseau, and Thomas Laurent. Learning tsp requires rethinking generalization. In *International Conference on Principles and Practice of Constraint Programming*, 2021.
- [4] Muhan Zhang, Pan Li, Yinglong Xia, Kai Wang, and Long Jin. Labeling trick: A theory of using graph neural networks for multi-node representation learning, 2022.
- [5] Jie Zhou, Ganqu Cui, Shengding Hu, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, Lifeng Wang, Changcheng Li, and Maosong Sun. Graph neural networks: A review of methods and applications, 2021.